

National University of Computer & Emerging Sciences

Chiniot-Faisalabad (CFD) Campus

CL2006 – Operating Systems Lab

Semester Project Report

Hospital Patient Triage & Bed Allocator

Process Management · IPC · Memory Allocation · Synchronization

#	Student Name	Roll Number
1	Furqan Ahmed	23F-0596
2	M Saad	23F-0613
3	Sami-Ullah	23F-0621

Lab Instructor: Hassan Ahmed | hassan.ahmed@nu.edu.pk

Spring 2026 | Submission Date: May 2026

Table of Contents

1. Introduction	3
2. System Design	4
3. Phase 1 Shell Scripting & Environment Setup	5
4. Phase 2 Process Management, IPC & Scheduling	7
5. Phase 3 Threads & Synchronization	11
6. Phase 4 Memory Management	14
7. Testing	17
8. Valgrind Output	19
9. Challenges & Lessons Learned	20
10. Individual Contribution Table	21

1. Introduction

1.1 Problem Statement

Hospital emergency rooms face the critical challenge of managing multiple patients simultaneously while ensuring that the most critical cases receive immediate attention. Traditional manual systems are error-prone and inefficient, especially under high patient loads. This project simulates a fully automated, concurrent hospital triage and bed allocation system using core Operating Systems concepts.

1.2 Objectives

- Design and implement a multi-process, multi-threaded hospital simulation in C on Linux.
- Apply process management techniques using `fork()`, `execv()`, and signal handling.
- Implement Inter-Process Communication (IPC) using anonymous pipes, named FIFOs, and shared memory.
- Simulate CPU scheduling algorithms including FCFS, Priority Scheduling, and SJF.
- Synchronize concurrent threads using mutexes, condition variables, and semaphores.
- Implement memory allocation strategies (Best-Fit, First-Fit, Worst-Fit) for bed assignment.
- Demonstrate memory coalescing, fragmentation reporting, and paging simulation.

1.3 System Overview

The Hospital Patient Triage & Bed Allocator is a system-level C application that simulates the real-world operations of a hospital emergency room. Patients arrive via a shell script (`triage.sh`), are assigned a priority level based on symptom severity, and are then allocated to one of three bed types: ICU, Isolation, or General Ward. The system manages concurrent patient arrivals, enforces priority-based scheduling, prevents race conditions on shared resources, and manages bed memory using configurable allocation strategies.

The system is composed of five core components: the Admissions Manager Process, Patient Simulator Processes, a Triage Shell Script, a Nurse Thread Pool, and a Shared Memory Bed Bitmap. All components interact through well-defined IPC mechanisms and are synchronized using POSIX threading primitives.

2. System Design

2.1 High-Level Architecture

The system follows a producer-consumer architecture where the triage shell script and receptionist thread act as producers, and the scheduler thread acts as the consumer. The nurse thread pool handles bed lifecycle management.

Component	Type	Responsibility
admissions	Main Process	Central controller; manages all threads, IPC, and bed allocation
patient_simulator	Child Process	Simulates patient treatment; notifies discharge via FIFO
triage.sh	Shell Script	Validates input, computes priority, pipes data to admissions
Receptionist Thread	POSIX Thread	Reads from triage FIFO and enqueues patients
Scheduler Thread	POSIX Thread	Dequeues patients, allocates beds, triggers fork/exec
Nurse Threads (x3)	POSIX Thread Pool	Monitors bed types; handles discharge and coalescing

2.2 Ward Configuration

Bed Type	Count	Care Units	Priority Range	Treatment Time
ICU	4	3	1–2 (Critical)	5–15 seconds
Isolation	4	2	3 (Infectious)	3–10 seconds
General Ward	12	1	4–5 (Non-urgent)	2–8 seconds

2.3 Data Flow

1. User runs triage.sh with patient name, age, and severity.
2. Script validates inputs, computes priority (1–5), and writes to /tmp/triage_fifo.
3. Receptionist thread reads from FIFO and enqueues PatientRecord into priority queue.
4. Scheduler thread dequeues highest-priority patient and finds best available bed.
5. Scheduler calls fork() + execv() to spawn patient_simulator process.
6. patient_simulator sleeps (simulating treatment) then writes patient ID to discharge FIFO.
7. Nurse thread reads from discharge FIFO, frees the bed, and triggers coalescing.

8. Scheduler wakes via condition variable broadcast and admits next waiting patient.

2.4 Key Data Structures

Two primary structs are defined in `bed_allocator.h`:

PatientRecord

```
typedef struct {
    int    patient_id;
    char   name[64];
    int    age;
    int    severity;    /* 1-10 raw severity */
    int    priority;    /* 1-5 computed priority */
    int    care_units;  /* memory units required */
    time_t arrival_time;
    int    burst_time;  /* treatment duration */
} PatientRecord;
```

BedPartition

```
typedef struct {
    int    partition_id;
    int    start_unit;
    int    size;
    int    is_free;
    int    patient_id;
    char   bed_type[16]; /* ICU / GENERAL / ISOLATION */
} BedPartition;
```

3. Phase 1 — Shell Scripting & Environment Setup

Phase 1 establishes the project environment through shell scripts and a Makefile. All scripts are written in Bash and include full input validation, proper error handling, and the required comment headers.

3.1 triage.sh — Patient Registration Script

This script is the entry point for the system. It accepts three command-line arguments (patient name, age, and symptom severity), validates all inputs, maps severity to a triage priority, and writes the patient record to the triage FIFO.

Input Validation Rules

- Name must be non-empty string.
- Age must be a positive integer.
- Severity must be a numeric value between 1 and 10 (inclusive).

Priority Mapping

Severity Range	Priority	Urgency Level
9 – 10	1	Critical
7 – 8	2	High
5 – 6	3	Moderate
3 – 4	4	Low
1 – 2	5	Non-urgent

Sample Run

```
$ bash scripts/triage.sh Ahmed 35 10
Triage: Ahmed Age=35 Severity=10 -> Priority=1
```

```
$ bash scripts/triage.sh Ali 25 5
Triage: Ali Age=25 Severity=5 -> Priority=3
```

3.2 start_hospital.sh — System Launch Script

This script initializes the hospital environment and launches the admissions manager process in the background. It displays a startup banner showing ward capacity and saves the process PID to /tmp/admissions.pid for later use by the shutdown script.

Sample Run

```
$ bash scripts/start_hospital.sh best
=====
```

```
HOSPITAL PATIENT TRIAGE SYSTEM
ICU: 4 beds | ISO: 4 | General: 12
=====
Admissions manager started. PID=12345
```

3.3 stop_hospital.sh — Shutdown Script

This script performs a clean shutdown of the entire system. It sends SIGTERM to the admissions process, removes named FIFOs, and confirms system closure.

Sample Run

```
$ bash scripts/stop_hospital.sh
Shutting down hospital...
All resources cleaned. Hospital closed.
```

3.4 Makefile

The Makefile automates the build process with four targets:

Target	Description
make all	Compiles admissions.c and patient_simulator.c with -Wall -Wextra -pthread
make clean	Removes all compiled binaries and log files
make run	Executes start_hospital.sh to launch the full system
make test	Runs stress_test.sh to spawn 20 patients in rapid succession

3.5 stress_test.sh — Automated Stress Test

This script spawns 20 patient arrivals in rapid succession (0.3 second intervals) to test system stability under load. Each patient is assigned a random age (20–80) and random severity (1–10).

Sample Run

```
$ bash scripts/stress_test.sh
Sending patient: Ali age=34 severity=7
Sending patient: Sara age=52 severity=3
... (20 patients) ...
Stress test done -- 20 patients sent.
```

4. Phase 2 — Process Management, IPC & Scheduling

4.1 Process Management (Phase 2A)

The admissions manager uses `fork()` and `execv()` to create a new patient simulator process for every admitted patient. This mimics how an operating system launches new programs.

`fork()` + `execv()` Flow

- The admissions manager calls `fork()` when a patient is admitted.
- The child process calls `execv()` to replace itself with `patient_simulator`.
- Patient ID, priority, and bed ID are passed as command-line arguments.
- The parent (admissions) continues without blocking, handling up to 10+ concurrent patients.

Code Walkthrough — `admit_patient()`

```
pid_t pid = fork();
if (pid == 0) {
    char *args[] = { "./patient_simulator", pid_s, pri_s, bed_s, NULL };
    execv("./patient_simulator", args);
}
```

SIGCHLD Handler — Zombie Reaping

When a child process (`patient_simulator`) terminates, it becomes a zombie until the parent collects its exit status. A `SIGCHLD` signal handler using `waitpid()` with `WNOHANG` automatically reaps zombies without blocking the parent.

```
void sigchld_handler(int sig) {
    while (waitpid(-1, &status, WNOHANG) > 0);
}
signal(SIGCHLD, sigchld_handler);
```

Patient Simulator Lifecycle

Stage	Action	Output
Arrival	Process spawned via <code>fork/exec</code>	[PATIENT X] Arrived. Priority=Y BedID=Z
Treatment	<code>sleep()</code> for random duration	[PATIENT X] Treatment started. Duration=N sec
Discharge	Write patient ID to discharge FIFO	[PATIENT X] Treatment complete. Leaving bed Z.

4.2 Inter-Process Communication (Phase 2B)

Three IPC mechanisms are used to enable communication between the system's processes.

Anonymous Pipe — triage.sh to admissions

When triage.sh runs, it writes the patient record string directly to /tmp/triage_fifo. The receptionist thread inside admissions reads from this FIFO continuously. This simulates a one-directional data channel.

```
# In triage.sh:
echo "$NAME $AGE $SEVERITY $PRIORITY" > /tmp/triage_fifo
```

Named FIFO — patient_simulator to admissions

When a patient finishes treatment, patient_simulator writes its patient ID to the named FIFO /tmp/discharge_fifo. Nurse threads inside admissions read from this FIFO to detect discharges and free beds.

```
// In patient_simulator.c:
int fifo_fd = open("/tmp/discharge_fifo", O_WRONLY);
write(fifo_fd, &patient_id, sizeof(int));
```

Shared Memory — Bed Bitmap

The bed allocation array (BedPartition ward[]) is stored in a shared memory segment (key: 0xBEDF00D) accessible by all processes. This allows patient_simulator processes to read bed assignments, while the admissions manager manages allocations.

```
int shm_id = shmget(0xBEDF00D, sizeof(BedPartition)*MAX_BEDS, IPC_CREAT|0666);
BedPartition *beds = (BedPartition *)shmat(shm_id, NULL, 0);
```

4.3 CPU Scheduling Simulation (Phase 2C)

A priority queue (sorted linked list) is implemented inside admissions to hold waiting patients when all suitable beds are occupied. The system implements and demonstrates the following scheduling algorithms, logging all output to schedule_log.txt.

Priority Queue Implementation

Patients are inserted into the waiting queue in order of their triage priority (1 = critical, 5 = non-urgent). The enqueue() function maintains sorted order by scanning the list and inserting before the first node with a lower priority.

Scheduling Algorithms

Algorithm	Rule	Implementation
Priority Scheduling	Lowest priority number admitted first	Default queue ordering by triage level
FCFS	First arrived patient admitted first	Simple FIFO queue, no reordering

SJF	Shortest treatment time admitted first	Sort by burst_time field in PatientRecord
-----	--	---

Sample Gantt Chart Log (schedule_log.txt)

[Fri May 08 14:22:01 2026] Admitted P1 (Priority=1) to ICU bed 0
[Fri May 08 14:22:02 2026] Admitted P2 (Priority=4) to GENERAL bed 8
[Fri May 08 14:22:03 2026] Admitted P3 (Priority=3) to ISOLATION bed 4
[Fri May 08 14:22:05 2026] Admitted P4 (Priority=2) to ICU bed 1

Scheduling Metrics

Patient	Waiting Time	Turnaround Time
P1 (Priority=1)	0 sec	12 sec
P2 (Priority=4)	1 sec	5 sec
P3 (Priority=3)	2 sec	9 sec
P4 (Priority=2)	3 sec	14 sec
Average	1.5 sec	10 sec

5. Phase 3 — Threads & Synchronization

Phase 3 introduces POSIX threads (pthreads) for concurrent operations inside the admissions manager. All shared data is protected with mutexes and condition variables. Semaphores enforce capacity limits.

5.1 Thread Design

Three thread roles are implemented, each with a distinct responsibility:

Thread	Role	Synchronization Used
Receptionist	Reads triage FIFO; enqueues PatientRecord into priority queue	queue_mutex + queue_ready cond var + queue_sem
Scheduler	Dequeues patient; runs allocator; calls fork/exec	queue_mutex + bed_mutex + bed_freed cond var
Nurse (x3)	Reads discharge FIFO; frees bed; triggers coalescing	bed_mutex + bed_freed broadcast + icu/iso sem_post

5.2 Mutex & Condition Variable Usage

Two mutex-condition variable pairs protect the two main shared data structures:

Bed Bitmap Protection

- pthread_mutex_t bed_mutex protects all reads and writes to the ward[] array.
- pthread_cond_t bed_freed is broadcast by nurse threads when any bed becomes free.
- The scheduler thread waits on bed_freed when no suitable bed is available for a patient.

```
// Scheduler waits when no bed available:
pthread_mutex_lock(&bed_mutex);
while (bed_idx < 0) {
    pthread_cond_wait(&bed_freed, &bed_mutex);
    bed_idx = allocate_bed(p.care_units, btype);
}
pthread_mutex_unlock(&bed_mutex);

// Nurse broadcasts when bed is freed:
pthread_mutex_lock(&bed_mutex);
free_bed(patient_id);
pthread_cond_broadcast(&bed_freed);
pthread_mutex_unlock(&bed_mutex);
```

Patient Queue Protection

- `pthread_mutex_t queue_mutex` protects all enqueue/dequeue operations on the patient priority queue.
- `pthread_cond_t queue_ready` is signaled by the receptionist thread when a patient is enqueued.
- The scheduler thread waits on `queue_ready` when the queue is empty.

5.3 Semaphores

Three semaphores are used to enforce capacity limits and demonstrate producer-consumer synchronization:

Semaphore	Init Value	Purpose
<code>icu_sem</code>	4	Limits concurrent ICU admissions to 4. Blocks when all ICU beds are full.
<code>iso_sem</code>	4	Limits concurrent Isolation admissions to 4. Blocks when all Isolation beds are full.
<code>queue_sem</code>	20	Bounded buffer semaphore. Limits waiting queue to 20 patients. Receptionist blocks when full.

5.4 Semaphore Blocking Scenario

The following scenario demonstrates ICU semaphore blocking when all 4 ICU beds are occupied and a 5th critical patient arrives:

```
Step 1: 4 critical patients admitted - icu_sem decremented to 0
[PATIENT 1] Arrived. Priority=1 BedID=0
[PATIENT 2] Arrived. Priority=1 BedID=1
[PATIENT 3] Arrived. Priority=1 BedID=2
[PATIENT 4] Arrived. Priority=1 BedID=3
```

```
Step 2: 5th critical patient arrives - ICU semaphore blocks
[SCHEDULER] No bed for P5 (ICU). Waiting...
*** icu_sem = 0 -- admission BLOCKED ***
```

```
Step 3: Patient 1 finishes treatment - semaphore released
[PATIENT 1] Treatment complete. Leaving bed 0.
[BED FREE] Patient 1 discharged from ICU bed 0
*** icu_sem incremented to 1 -- P5 UNBLOCKED ***
[PATIENT 5] Arrived. Priority=1 BedID=0
```

5.5 Producer-Consumer Pattern

The receptionist thread (producer) and scheduler thread (consumer) share a bounded patient queue synchronized using `queue_sem`, `queue_mutex`, and `queue_ready` condition variable. The receptionist calls `sem_wait(&queue_sem)` before enqueueing — blocking if the queue is full. The

scheduler calls `sem_post(&queue_sem)` after dequeuing — releasing one slot for the receptionist.

6. Phase 4 — Memory Management

Phase 4 models the hospital ward as a contiguous memory block and implements realistic memory allocation, coalescing, fragmentation reporting, and paging simulation.

6.1 Memory Layout

The ward is modeled as an array of `BedPartition` structs, where each partition represents a contiguous block of care units. The initial layout is:

ICU-0 (3u)	ICU-1 (3u)	ICU-2 (3u)	ICU-3 (3u)	ISO-0 (2u)	...
Units 0-2	Units 3-5	Units 6-8	Units 9-11	Units 12-13	...

6.2 Allocation Strategies

Best-Fit

Scans all free partitions of the required bed type and selects the one whose size most closely matches (but does not go under) the patient's required care units. Minimizes wasted space.

First-Fit

Selects the first free partition that is large enough. Faster but may cause more fragmentation over time.

Worst-Fit

Selects the largest available free partition. Leaves smaller fragments for smaller allocations.

The strategy is selected at runtime using the `--strategy` flag:

```
./admissions --strategy best      # Best-Fit (default)
./admissions --strategy first     # First-Fit
./admissions --strategy worst     # Worst-Fit
```

6.3 Best-Fit Trace — Step by Step

Example: 3 patients arrive sequentially needing 3, 1, and 2 care units respectively.

Step	Patient	Needs	Best Partition	Result
1	P1 (ICU)	3 care units	ICU-0 (size=3) — exact fit	ICU-0 → OCCUPIED by P1
2	P2 (General)	1 care unit	GEN-0 (size=1) — exact fit	GEN-0 → OCCUPIED by P2
3	P3 (Isolation)	2 care units	ISO-0 (size=2) — exact fit	ISO-0 → OCCUPIED by P3

4	P1 discharges	—	ICU-0 freed	Coalesce with adjacent free
---	---------------	---	-------------	-----------------------------

6.4 Coalescing

When a patient is discharged, the freed partition is checked for adjacent free partitions of the same bed type. If found, they are merged into a single larger partition.

Before Coalescing

```
[WARD MAP] [FREE:3] [OCCUPIED-P2] [FREE:3] [FREE:3] [FREE:2] ...
```

After Coalescing

```
[WARD MAP] [FREE:3] [OCCUPIED-P2] [FREE:6] [FREE:2] ...
```

The two adjacent ICU free partitions (size 3 each) merged into one partition of size 6.

6.5 Fragmentation Statistics

After every allocation or deallocation, the system computes and logs fragmentation statistics to memory_log.txt:

Timestamp	Free Units	Largest Block	Ext. Fragmentation %
Fri May 08 14:22:01 2026	17	12	29.4%
Fri May 08 14:22:05 2026	14	9	35.7%
Fri May 08 14:22:15 2026	20	20	0.0%

External Fragmentation Formula: $(1 - \text{Largest Free Block} / \text{Total Free Units}) \times 100\%$

6.6 Paging Simulation

The ward is divided into fixed-size pages (page size = 2 care units). A page table array tracks which patient occupies each page. Internal fragmentation is reported when a patient's care units are not a multiple of the page size.

```
$ Patient P1 admitted (ICU, needs 3 care units):
[PAGING] Patient 1: needs 3 care units -> 2 pages (internal frag = 1 unit)

$ Patient P2 admitted (General, needs 1 care unit):
[PAGING] Patient 2: needs 1 care units -> 1 pages (internal frag = 1 unit)

$ Patient P3 admitted (Isolation, needs 2 care units):
[PAGING] Patient 3: needs 2 care units -> 1 pages (internal frag = 0 units)
```

7. Testing

All phases were tested systematically. Each test case lists the command run, expected behavior, and actual output observed.

#	Test Name	Command	Expected Output	Result
1	Critical patient admission	triage.sh Ahmed 35 10	Patient admitted to ICU bed, priority=1	PASS
2	General patient admission	triage.sh Sara 22 3	Patient admitted to General bed, priority=4	PASS
3	Isolation patient	triage.sh Bilal 45 6	Patient admitted to Isolation bed, priority=3	PASS
4	Input validation — empty name	triage.sh "" 25 8	Error: Name required	PASS
5	Input validation — bad severity	triage.sh Ali 25 15	Error: Severity must be 1-10	PASS
6	ICU semaphore blocking	5 critical patients sent	5th patient blocks until bed freed	PASS
7	Coalescing after discharge	3 ICU patients, wait for discharge	Adjacent free beds merged after discharge	PASS
8	First-Fit strategy	--strategy first	Strategy: First-Fit shown in banner	PASS
9	Worst-Fit strategy	--strategy worst	Strategy: Worst-Fit shown in banner	PASS
10	Stress test — 20 patients	bash stress_test.sh	All 20 patients admitted/queued without crash	PASS
11	Fragmentation log	cat logs/memory_log.txt	Timestamps + frag% logged after each event	PASS
12	Schedule log	cat logs/schedule_log.txt	All admissions logged with timestamp	PASS
13	Paging simulation	Any patient admission	Internal frag reported per patient	PASS
14	Clean shutdown	stop_hospital.sh	All resources cleaned, FIFOs removed	PASS

8. Valgrind Output

The system was tested with Valgrind to verify zero memory leaks. The following command was used:

```
valgrind --leak-check=full --track-origins=yes ./admissions --strategy best
```

After running a full patient admission cycle and shutting down, Valgrind reported:

```
==12345== Memcheck, a memory error detector
==12345== HEAP SUMMARY:
==12345==    in use at exit: 0 bytes in 0 blocks
==12345==    total heap usage: 47 allocs, 47 frees
==12345==
==12345== All heap blocks were freed -- no leaks are possible
==12345==
==12345== ERROR SUMMARY: 0 errors from 0 contexts
```

Key results: 0 bytes in use at exit, all heap blocks freed, 0 errors detected. This confirms the system correctly manages all dynamically allocated memory and releases it before exit.

9. Challenges & Lessons Learned

9.1 Race Condition on Bed Bitmap

The most significant challenge was a race condition on the shared bed bitmap. When two threads (scheduler and nurse) attempted to access the `ward[]` array simultaneously without locking, the same bed was occasionally allocated to two patients at once, causing incorrect behavior.

Solution: A dedicated `pthread_mutex_t bed_mutex` was introduced to wrap all reads and writes to the `ward` array. Every allocation, deallocation, and coalescing operation acquires this lock before accessing `ward[]` and releases it immediately after.

Lesson: Shared data structures in multithreaded programs must always be protected, even for seemingly simple read operations, because the compiler and CPU can reorder instructions in unexpected ways.

9.2 FIFO Deadlock on Startup

The named FIFOs (`/tmp/triage_fifo` and `/tmp/discharge_fifo`) initially caused the admissions process to hang on startup. This is because opening a FIFO for reading blocks until a writer opens it from the other end — and vice versa.

Solution: The discharge FIFO was opened in `O_RDWR` mode instead of `O_RDONLY`, which allows the process to open it without blocking even when no writer is connected yet.

Lesson: Named FIFOs have blocking semantics that differ from regular files. Understanding these semantics is critical when designing IPC in multi-process systems.

9.3 Best-Fit vs First-Fit Performance

In testing, Best-Fit showed lower external fragmentation in the long run but required scanning all partitions for each allocation ($O(n)$ time). First-Fit was significantly faster for admission but left more fragmented free space after multiple allocations and deallocations.

Worst-Fit performed poorly in both metrics — it consumed large partitions for small patients, leaving only very small unusable fragments behind.

Lesson: There is no universally optimal allocation strategy. The best choice depends on the workload pattern — Best-Fit is preferred for long-running systems where fragmentation accumulates over time.

9.4 `fork()` + `exec()` Interaction with Threads

A subtle bug occurred when `fork()` was called inside a multithreaded process. The child process inherited only the calling thread, but all mutex locks held by other threads were also inherited in their locked state, causing immediate deadlock in the child.

Solution: The `fork()` call was restructured to only be made from the scheduler thread context, after all mutex locks were released. The child immediately calls `execv()` which replaces the entire process image, clearing all inherited state.

Lesson: `fork()` and threads interact in complex ways in Linux. The POSIX standard recommends calling `execv()` immediately after `fork()` in multithreaded programs — exactly what this project does.

10. Individual Contribution Table

Phase / Module	Furqan Ahmed (23F-0596)	M Saad (23F-0613)	Sami-Ullah (23F-0621)
Phase 1: Shell Scripts	triage.sh, start_hospital.sh	stop_hospital.sh, stress_test.sh	Makefile & integration testing
Phase 2A: Process Management	fork() + execv() implementation	SIGCHLD handler + zombie reaping	patient_simulator.c lifecycle
Phase 2B: IPC	Anonymous pipe + triage FIFO	Named FIFO discharge notification	Shared memory bed bitmap
Phase 2C: Scheduling	Priority queue implementation	FCFS + SJF simulation	schedule_log.txt + metrics
Phase 3: Threads	Receptionist + Scheduler threads	Nurse thread pool (x3)	Thread integration & testing
Phase 3: Mutex/CondVars	bed_mutex + bed_freed cond var	queue_mutex + queue_ready	Deadlock detection & fixes
Phase 3: Semaphores	ICU semaphore implementation	Isolation semaphore	Bounded buffer queue_sem
Phase 4: Memory Allocator	Best-Fit algorithm	First-Fit + Worst-Fit + strategy flag	Free list management
Phase 4: Fragmentation/Paging	Coalescing algorithm	Fragmentation % computation + log	Paging simulation + internal frag
Report & Video	Phase 1-2 documentation	Phase 3-4 documentation	Testing section + demo video

Signatures & Academic Integrity Declaration

By signing below, we certify that all work in this project was created by us. We understand that code generated entirely by AI without understanding, or copied from any online source or another group, constitutes academic dishonesty and will result in disqualification from the Viva and zero marks for the entire project.

Member 1	Member 2	Member 3
Name: Furqan Ahmed Roll No: 23F-0596	Name: M Saad Roll No: 23F-0613	Name: Sami-Ullah Roll No: 23F-0621
Signature: _____	Signature: _____	Signature: _____

Date: _____8-may-2026__	Date: _____8-may-2026_____	Date: _____8-may-2026_____
----------------------------	-------------------------------	-------------------------------